

BASIC - basic principles of FRED.

FRED can operate in two different modes: *text input mode* and *command mode*. In text input mode, you enter the material you want to edit. In command mode, you enter commands that tell FRED to perform various editing actions.

Internally, FRED uses the full upper and lower case ASCII character set to store all its characters. However, it can read and write both ASCII and BCD card-image files.

FRED commands are one or two characters long. Whenever possible, a command's name suggests what the command does. Commands may be typed in either upper case or lower case. Many commands may be preceded by line addresses to indicate that the command should be applied to a particular line or set of lines. Many commands may be followed by one or more command arguments.

In general, any number of commands may be typed on a single line, but there are a few exceptions to this rule. In particular, there are a few commands which can only appear at the end of a line (so they can't be followed by other commands).

Commands typed on the same line may be separated by any number of spaces or tab characters. If there is no ambiguity of meaning, commands need not be separated at all.

The NL Character:

FRED generally uses the ASCII linefeed character to separate lines of text. We will represent this character as LF. Most ASCII terminals have a "linefeed" key to enter LF.

The carriage return character (written CR) is also treated as a LF when read from a file or from the terminal keyboard. We will use the term *new-line character* (written NL) in situations where you can use either CR or LF.

Since CR is the character that tells TSS a line of input is ready for processing, you will usually use CR as your new-line character.

As far as FRED is concerned, you may always use the linefeed as a NL character instead of the carriage return. However, TSS will accept no more than 160 characters including linefeeds between CRs, so you must use the CR occasionally. Furthermore, the "line delete" used by TSS (ctrl-X) destroys everything from the end of your current input line back to the previous CR including any intervening LFs you may have used to separate lines for FRED.

You should also note that the "character delete" of TSS can delete a LF just like any other character.

You can place a true CR character into a buffer's text by using its octal equivalent (\015). LF characters however cannot be placed literally into a buffer, since FRED interprets all LF characters as line delimiters.

The escape sequence "\;" stands for a NL. This is useful in situations where an actual new-line cannot be used or would be misinterpreted. For example, if you want to specify a new-line in some argument on the command line that invokes FRED, you can use \;.

Buffers and Buffer Names:

All text used by FRED must be stored in a *buffer*. A buffer is simply a set of text lines identified by a name. It might help to think of a buffer as a looseleaf notebook in which you enter and edit material.

Buffer names are strings of up to 16 characters. Any character may appear in a buffer name, but you have to give certain characters special treatment. In particular, you must type \C before NL characters, closing

parentheses ")", and special stream directive characters. (The special stream directive characters and \C are described in "[expl fred stream](#)").

Buffer names must be enclosed in parentheses unless they are exactly one character long and the 0-I(option is in effect. This option is explained in "[expl fred oi](#)".

FRED doesn't usually distinguish between upper and lower case letters in buffer names; thus (ab), (AB), (Ab), and (aB) all refer to the same buffer. FRED will pay attention to the case of letters that are immediately preceded by a \C. Thus (\CB) is the name of a buffer (B) as distinct from a buffer (b). Some other examples of valid buffer names are (), (war & peace), (w or k) and (.).

FRED lets you have any number of buffers you might need. (Having several buffers is like having several notebooks.) At any given time, one buffer has the special status of being the *current buffer*. To continue the comparison with notebooks, this is the notebook that you have open at the moment. There are some commands that allow you to copy or move text from one buffer to another (like taking a page out of one notebook and putting it into another) but most commands only work on one buffer at a time.

The B command tells FRED to go to a particular buffer (just like opening a notebook). The B command is described in "[expl fred b](#)". When you first enter FRED, the current buffer is one named (0).

In every buffer, one line is designated as the *current line*. Think of this as the line you are currently working with. Commands that modify a single line will modify the current line unless you explicitly specify a different line. The current line is referred to with the character "." (called "dot"). The value of "." is changed by many FRED commands.

Inputting Text:

This section describes a number of ways to put text into a buffer.

- The A, C, and I commands put FRED into text input mode. Once one of these commands is issued, all lines typed in are stored in the current buffer.

When you want to stop entering text, type \F. When FRED sees this special sequence, you will go back to command mode. The \F is not included in the text itself; it merely serves to indicate the end of the text.

- You can put text into a buffer by reading it from a temporary or permanent file. When a file is read into a buffer, FRED associates the file name with that buffer.

If changes are made to text that was read from a file, we say that the buffer is *questionable* until the text is saved in a file or a new file is read in. FRED assumes that you would not change the contents of a buffer unless you want the modified contents to be preserved, and the only way to preserve a buffer's contents is to write the buffer out to a file explicitly. FRED never files text automatically; the contents of every buffer vanish when FRED terminates execution, and thus all text is lost unless it has been explicitly written out to a file.

- The last way to put text into a buffer is to get it from another buffer. There are several commands to move or copy text from one buffer to another.

Memory Requirements for FRED:

FRED does not usually store the entire contents of every buffer in memory. Instead, it keeps a small amount of working space in memory and stores the contents of the buffers in a temporary work file. This file is called *qn*, where *n* is 0, 1, 2, or 3, depending on the level of the TSS callss stack. If there is an I/O error on this work file during FRED's operation, or if the work file is removed from the AFT, FRED will issue a short message describing the problem and terminate with a DRL ABORT. In this case, the contents of all buffers are lost if they have not been previously written out to a file.

Occasionally, FRED will find itself unable to grow its work file. In this case, an error message will warn you that one level of command voiding is permanently gone (for an explanation of command voiding, see "[expl fred v](#)").

By default, FRED requires three *pages* of memory as working space (where a page is 4096 words long). This number of working pages can be changed using the `O+B` or `O-B` option commands. The number of pages specified by these options represents a maximum; if FRED does not require as many pages as you have specified, it will only take as much memory as it needs.

Note that using `O+B` to increase your memory size will only be of use if you add enough memory to hold the entire file you're editing.

Number Registers:

In order to perform simple arithmetic operations, FRED lets you define number registers. Every number register has a name and contains a positive or negative integer. Register names follow the same syntax rules as buffer names.

Number registers are created and manipulated using the `N` command. The value of a number register may be placed into the input stream by using the `\N` construction. See "[expl fred escapes](#)" for more information.

FRED will let you use the same name for a buffer and a number register.

The Count Register:

The count register is a special number register denoted by the symbol `#`. FRED uses the count register to provide various kinds of numeric information. For example, when a file is read into a buffer, FRED sets the count register to the number of llinks that were read. The count register can also be set explicitly with the `N` command.

The contents of the count register may be displayed using the `#` command.

The Condition Register:

The condition register is an internal feature of FRED. It may be set either `TRUE` or `FALSE`. A number of commands set this register depending on the outcome of their actions.

The condition register is seldom used for ordinary editing tasks, but it's useful when writing programs. The `J` and `U` commands can test the contents of the condition register and use the result to control the flow of a program.

Init Files:

When FRED is first invoked from system level, it searches for a file named

`userid/fred/.init`

(where `userid` is your own `userid`). If such a file exists, FRED will execute the commands in that file before taking further action. The file `/fred/.init` is called an *init file*.

Init files are handy for setting options before an editing session. For example, if you prefer to do without prompting for text input, your init file can set the appropriate option every time you enter FRED. In this way, you can avoid typing the same set of commands at the beginning of each editing session.

Your init file can also be used to create a standard set of buffers, to define frequently used patterns, or to set up a collection of number registers.

If FRED does not find an init file in the current catalog, it will set its own default options and proceed with normal processing.

Language Limitations:

When FRED is writing a file, it will not create records longer than 2000 characters. If FRED reads in a line longer than 2000 characters, the line will be broken up into separate lines of 2000 characters each. The last of these lines will have a new-line character on the end but the others will not. Thus when you write the buffer out again, the component lines will be "reassembled" into one long line with a single terminating new-line.

FRED will not let an S command lengthen any line beyond 2400 characters.

The maximum amount of text that FRED can copy into the hidden buffer of a G or U command is 4000 bytes.

Copyright © 1998, Thinkage Ltd.